

Which Database we will use ?

In selecting the appropriate database architecture, a relational database management system (RDBMS) such as SQL is deemed essential for tables housing critical information, like customer, restaurant, and agent details. This choice is made because of the necessity for high consistency, ensuring swift and accurate reflection of any alterations, such as changes in payment details or menu prices.

SQL's adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties makes it particularly well-suited for maintaining data integrity and ensuring reliability in such scenarios.

Conversely, for data that exhibits more relaxed consistency requirements, such as restaurant and agent ratings, a NoSQL database is a judicious choice.

NoSQL, characterized by BASE (Basically Available, Soft state, Eventually consistent) properties, aligns with the nature of these datasets. This allows for greater flexibility, accommodating the dynamic and non-critical nature of rating information, where immediate and stringent consistency is not a primary concern.

This hybrid approach harnesses the strengths of both SQL and NoSQL databases, tailoring the database architecture to the specific needs and characteristics of the data being managed.

Communicating with the server

Employing RESTful APIs for the Restaurant Management System proves advantageous due to their simplicity, uniform interface, and statelessness. REST's straightforward design, using standard HTTP methods, facilitates ease of use and development, promoting interoperability across various platforms and programming languages.

- The resource-centric approach aligns well with entities in the system, and the statelessness enables horizontal scalability, crucial for handling the anticipated high traffic in a restaurant management system.
- Additionally, RESTful APIs support flexibility in data formats, standardized error handling through HTTP status codes, and caching mechanisms, contributing to efficient communication, error identification, and improved performance.

1. User Registration

1. Endpoint:

```
POST /api/users/register
```

2. Request:

```
{
  "username": "salik",
  "password": "secure_password",
  "email": "\"xyz@gmail.com\"",
  "address": "123 Main St, City",
  "phone": "+1234567890"
}
```

3. Response:

```
{
  "message": "Login successful",
  "userId": "12345",
  "token": "authentication_token"
}
```

2. User Authentication

1. Endpoint:

```
GET /api/restaurants/search?location=City&cuisine=Italian&rating=4
```

2. Request:

```
{
  "username": "salik",
  "password": "secure_password"
}
```

3. Response:

```
{
  "message": "Login successful",
  "userId": "12345",
  "token": "authentication_token"
}
```

3. Restaurant Search and Order Placement

1. Search Nearby Restaurants

Endpoint:

End point

```
GET /api/restaurants/search?location=City&cuisine=Italian&rating=4
```

Response:

```
[
  {
    "restaurantId": "abc123",
    "name": "Italian Delight",
    "location": "456 Main St, City",
    "cuisine": "Italian",
    "rating": 4.5  },
  {
    "restaurantId": "def456",
    "name": "Pizza Palace",
    "location": "789 Main St, City",
    "cuisine": "Italian",
    "rating": 4.2  }
]
```

2. Place an Order

Endpoint:

Endpoint:

endpoints

POST /api/orders/place

Request:

```
{
  "userId": "12345",
  "restaurantId": "abc123",
  "items": [
    { "itemId": "item1", "quantity": 2 },
    { "itemId": "item2", "quantity": 1 } ],
  "deliveryPreference": "delivery", "paymentMethod": "credit_card"
}
```

Response:

```
{
  "orderId": "order123",
  "message": "Order placed successfully",
  "totalAmount": 25.99
}
```

Response

3. Restaurant Management(View Orders for a Restaurant):

Endpoint:

GET /api/restaurants/orders?restaurantId=abc123

Response:

```
[
  {
    "orderId": "order123",
```

```
    "userId": "user456",  
  "items": [  
    {  
      "itemId": "item1",      "quantity": 2    },  
    {      "itemId": "item2",      "quantity": 1    }  
  ],  
  "totalAmount": 25.99,  
  "status": "pending" },  
  // Additional orders...  
]
```

4. Delivery and Real-time Tracking(Track Delivery Order):

Endpoint:

```
GET /api/delivery/track?orderId=order123
```

Response:

```
{  
  "orderId": "order123",  
  "status": "in_transit",  
  "location": {    "latitude": 37.7749,  
    "longitude": -122.4194  
  }  
}
```